WEB DEVELOPMENT · PROJECT 7

NASA Space Explorer

Project submission & reflections

> **Reflection draft for review.** These answers describe the completed project. Review the first-person wording before submitting.

1. Website URL

https://33misha.github.io/07-nasa-space-explorer/

Expected URL — deployment requires verification. Confirm this address in GitHub Settings → Pages, then open it in a private browser. Replace this note with the confirmed deployment result before submitting. The required link is the public website, not the repository or Codespace.

1B. LevelUps completed

- **APOD videos:** Video entries stay in the nine-day gallery with a preview or fallback and a clear way to watch.
- **Random space fact:** A “Did You Know?” section randomly selects a fact on page load.
- **Hover zoom:** Gallery images scale smoothly on hover, with reduced-motion preferences respected.

2. Working with a real API

> What was it like using NASA's APOD API in your project? What challenges did you face when trying to connect, fetch data, or handle the results?

Working with NASA's APOD API turns this app from a static page into something that responds to real information. A visitor's dates become request parameters, and the returned objects supply the gallery's titles, dates, media, and explanations. One lesson is that fetching data and displaying useful data are separate jobs. A response must be checked before it can be trusted to represent the selected window.

The nine-day requirement makes that especially clear. The start and end dates are inclusive, so they must be eight calendar days apart. Using calendar dates also avoids off-by-one errors around daylight saving changes. After the request finishes, the app checks that the expected dates are actually present. A day that NASA has not published yet needs an understandable message, not a silent eight-card result.

APOD is also not a uniform collection of image files. Some days are videos, image fields can differ, and an entry may point to a NASA article. An upstream migration caused the legacy API to return generic NASA logos titled “NASA Science.” The app detects those records and requests the same dates from NASA Science's official APOD endpoint. Loading, network failures, API-key errors, and quota limits need clear feedback as well. I used AI assistance to help implement and review these cases while keeping the app's behavior tied to the assignment and actual API fields. A dependable API app needs validation and recovery as well as a successful fetch.

3. Debugging interview plan

> How do you approach debugging when something doesn't work as expected? Think about something in this project that did not work, and explain how you would identify and fix the cause.

My answer would start with a repeatable process: reproduce the problem, state the expected result, inspect the evidence, isolate the cause, make a focused change, and check the result. I would use the browser Console to find JavaScript errors and the Network panel to see whether a request was sent, which parameters it used, and what NASA returned. This separates a broken click handler from a failed request or a rendering problem.

A concrete example from development is an upstream response that looked successful but contained the wrong content. The legacy APOD endpoint returned nine records titled “NASA Science,” pointing to a NASA logo. Counting items alone would have passed the gallery check. Comparing the response titles and media URLs with the requested astronomy content exposed an upstream migration, rather than a CSS defect. The app detects those generic records and requests the same dates from NASA Science's official APOD endpoint. Development checks confirmed that endpoint could return nine real entries, including a video in a historical window.

I would then check a normal historical range and deliberately test an invalid range, a video day, an incomplete response, and a failed request. For an eight-card result, I would compare requested dates with returned dates before changing layout code. After a fix, I would repeat the original reproduction and one nearby case to look for regressions. In an interview, I would distinguish development checks performed with AI assistance from checks I personally ran, and describe only results I can explain from the evidence.

4. LinkedIn post draft

> Reflect on what you built and learned, what surprised you about APIs, and how AI supported your process. Publishing the post is optional.

I built a NASA Space Explorer using HTML, CSS, and JavaScript! The app connects to NASA's Astronomy Picture of the Day API and turns a selected nine-day window into a gallery of space images and videos. Each entry opens a detailed view with its title, date, and explanation.

One lesson from this project: a successful API request is only the beginning. Date boundaries, different media types, loading states, missing data, and quota limits all affect whether an app feels reliable. I also added a random space fact and smooth image zoom, with keyboard access and responsive layouts in mind.

I used AI assistance to support the implementation and review process. The project gave me a concrete way to connect API data, DOM updates, CSS, and debugging decisions in one application.

Project link after deployment verification:
https://33misha.github.io/07-nasa-space-explorer/

#WebDevelopment #JavaScript #NASA #APIs #LearningInPublic

Before submission: personalize these drafts, verify the live GitHub Pages URL, update its status, and export this document using the browser's Print → Save as PDF option. Upload the completed PDF to the Project 7 submission page on HQ. This document does not publish a LinkedIn post or submit coursework automatically.